

Introduction

In the context of Industry 4.0, unexpected faults in rotating equipment regularly occur, causing critical production downtime. To ensure better productivity, this project aims to integrate Artificial Intelligence for the early diagnosis and prognosis of faults through the analysis of **vibration** data, facilitating a transition toward advanced **predictive maintenance** strategies.

Motivation:

- Unexpected production downtime represents a critical cost for manufacturers.
- Deploying Machine Learning models via *Edge Computing* (specifically **TinyML** on microcontrollers) allows data analysis right at the sensor level, thereby reducing latency, minimizing cloud dependency, and securing sensitive data.

Methodology: TinyML on ESP32

Embedded Processing Flow:

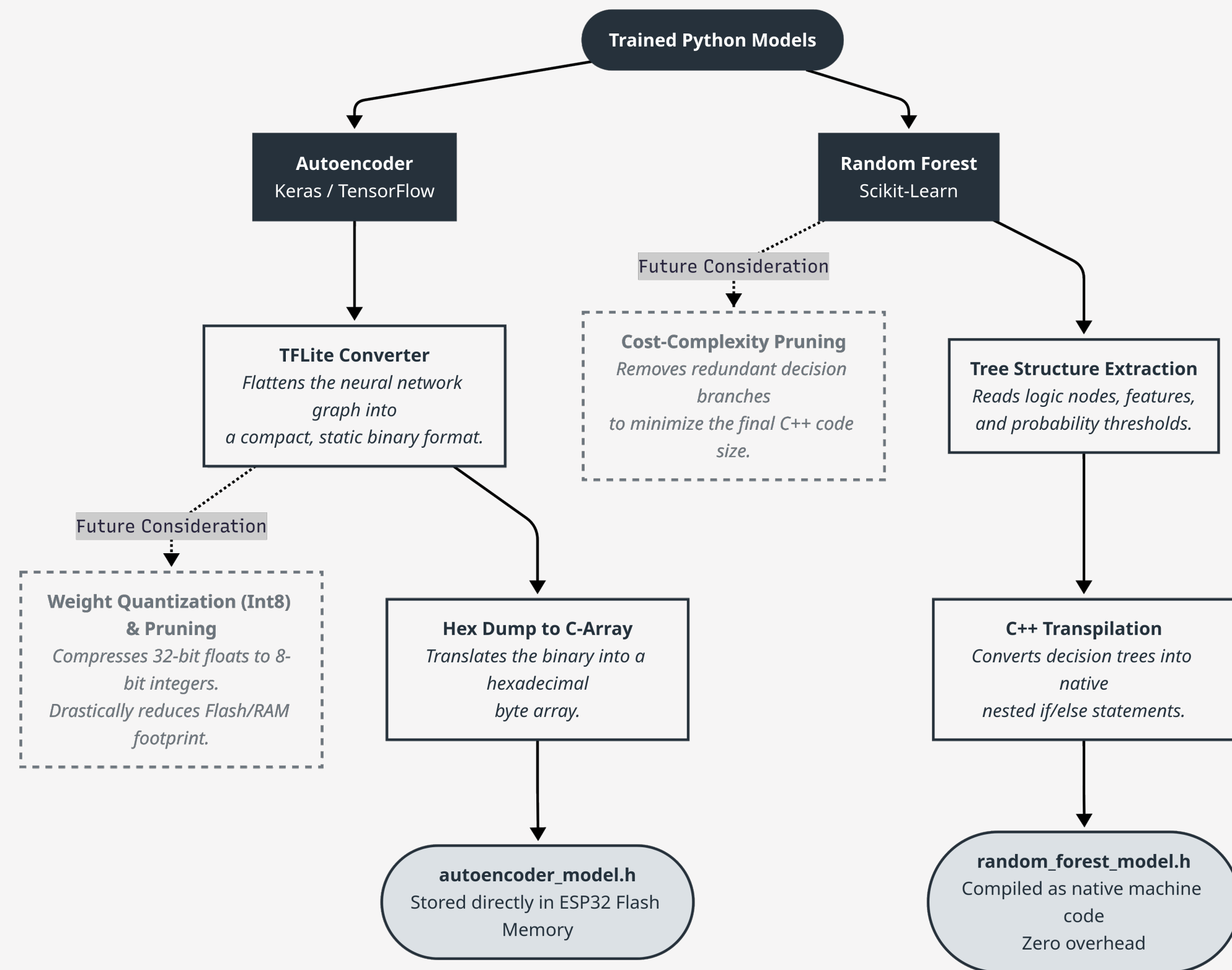


Fig. 3 — TinyML model optimization and deployment pipeline.

Transformations and Optimizations:

- **Implemented:** The Autoencoder is flattened into a static binary format and exported as a direct C-array via TFLite Micro. The Random Forest is transpiled into native C++ nested if/else rules, eliminating any framework overhead.
- **Transformations if needed:** **Weight Quantization (Int8)** for the Autoencoder (32-bit floats into 8-bit integers) and **Cost-Complexity Pruning** for the Random Forest to remove redundant tree branches.

ESP32 Constraints:

~520 KB SRAM, 4 MB Flash, 240 MHz CPU.

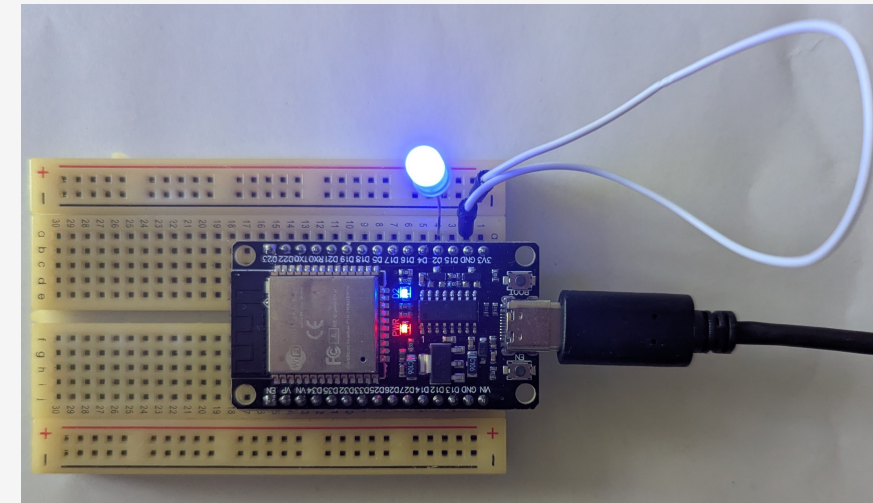


Fig. 4 — ESP32

First Results (on healthy and faulty files)

Model Proposal ("Both"): Combining the Autoencoder to block unknown anomalies and the Random Forest to provide a precise diagnosis (identifying the specific defective part) once the anomaly is confirmed.

Table 1: Timing Performances - Case Western Dataset (CWRU)

Model	Train Time	Inf. PC	Inf. ESP32	Accuracy
Random Forest	30.96 s	6.68 ms	3.82 ms	94.60 %
Autoencoder	42.58 s	58.09 ms	58.24 ms	58.50 %
Both (AE+RF)	73 s	63 ms	67.51 ms	94.53 %

Table 2: Timing Performances - MaFaulDa Dataset

Model	Train Time	Inf. PC	Inf. ESP32	Accuracy
Random Forest	6.21 s	7.36 ms	3.31 ms	99.80 %
Autoencoder	27.23 s	71.09 ms	58.22 ms	70.64 %
Both (AE+RF)	39.15 s	74 ms	77.01 ms	99.00 %

* Notes: Training on 60% of samples, Accuracy on 20%. Inference times show the worst-case for a single sample.
Inf. PC : CPU Intel Core i7-7500U, 8 GB RAM (Linux Mint)

Table 3: Models Characteristics

Model	Complexity	Parameters	Operations
Random Forest	$O(N_{trees} \times D)$	55 678	804 Ops
Autoencoder	$O(W \times C)$	266 385	573.4 kFLOPs
Both (AE+RF)	Variable	322 063	573.4 kFLOPs (+ 804 Ops max)

* N_{trees} : Number of trees, D : Maximum tree depth, W : Number of weights, C : Computational cost per weight.

Context

Definitions:

- **Vibration:** Mechanical oscillation of a component around an equilibrium point. In industry, an alteration in the vibratory signature is the first physical symptom of hardware degradation.
- **Predictive Maintenance:** A strategy consisting of real-time equipment monitoring to predict and schedule interventions *before* critical failure occurs, thus optimizing machine lifespan.
- **Bearing:** An essential mechanical component that reduces friction between moving parts. It consists of four main parts prone to wear: the inner race, the outer race, the balls and the cage.

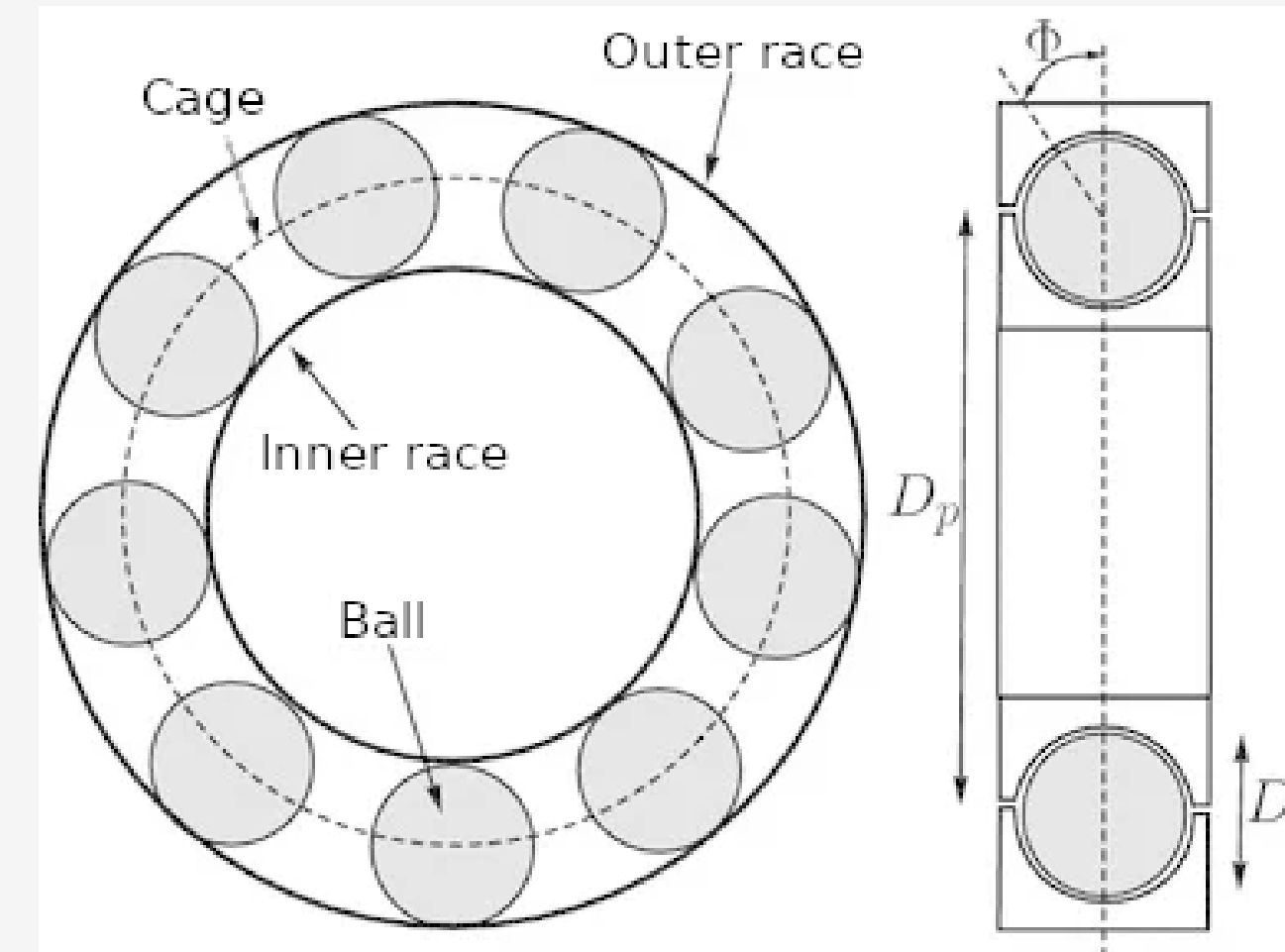


Fig. 1 — Internal structure of a ball bearing and potential failure zones

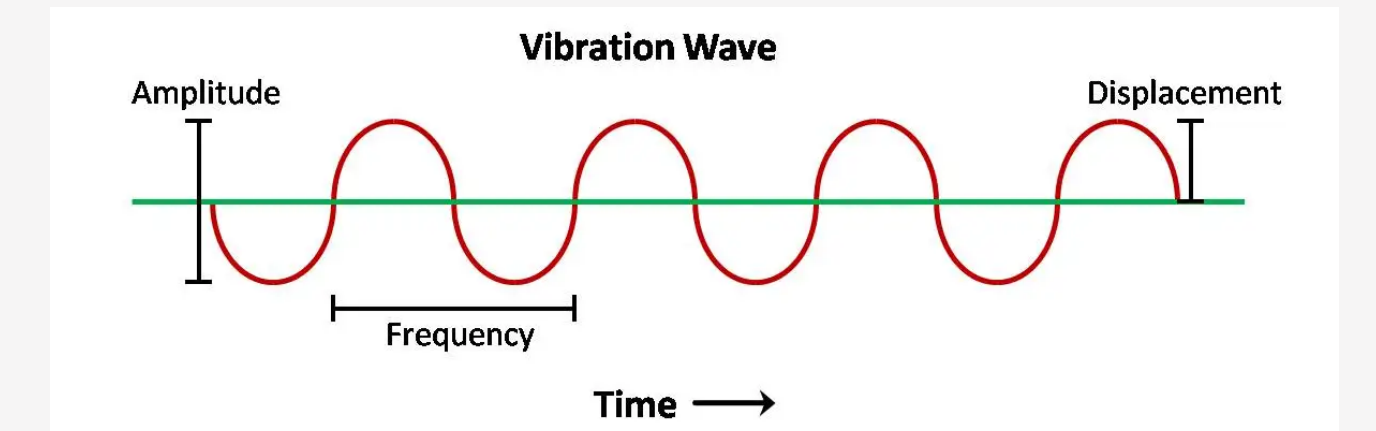


Fig. 2 — Explanation of a vibration

Experimentation

Studied Datasets:

- **Case Western (CWRU)**^[1]: Artificial lab faults (EDM) Distinct signatures, Easy detection.
- **MaFaulDa**^[2]: Mechanical simulator Complex intertwined wear, Subtle signatures, Realistic & Hard.

Supervised vs. Unsupervised:

- **Supervised (Random Forest):** Learns predefined classes only. Precise diagnosis for known conditions.
- **Unsupervised (Autoencoder)**^[3]: Learns to compress and reconstruct data. High error (MSE) detects unseen anomaly.

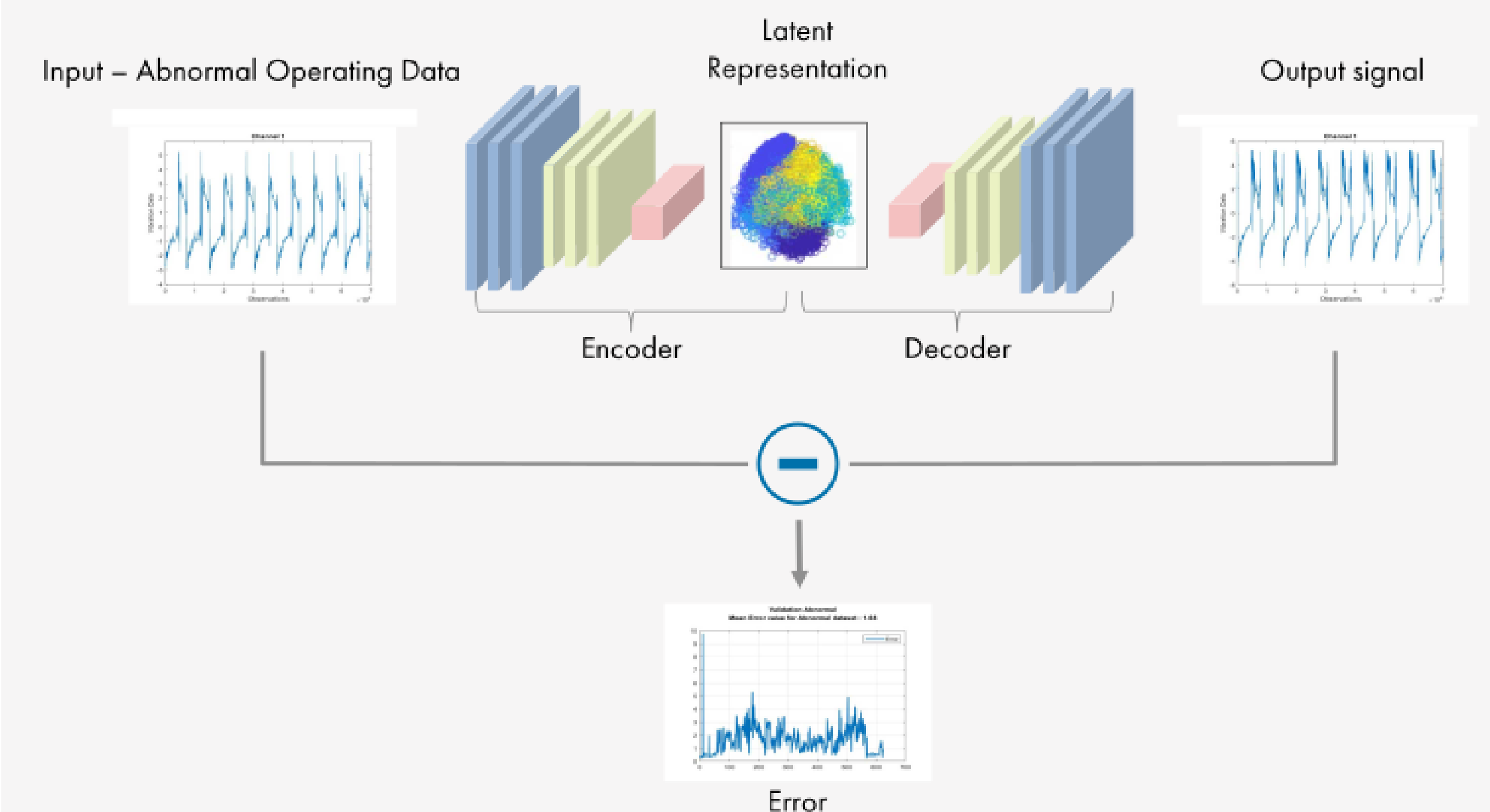


Fig. 5 — Architecture of an Autoencoder ^[4]

Conclusion & Perspectives

Conclusion:

- **TinyML Feasibility:** The ESP32 successfully runs complex ML pipelines (Autoencoder + Random Forest) locally without extreme model degradation.
- **Deployment Success:** TFLite Micro and native C++ transpilation enabled a seamless transition from Python to embedded hardware.
- **Performance:** Fast inference (~100 ms) and minimal memory footprint prove *Edge AI* is a viable, real-time solution for Industry 4.0.

Perspectives:

- Prove the viability of the model proposal by testing its resilience against previously unseen faults in real-world conditions.
- Maximize the optimization of the models implementation on the ESP32, specifically through advanced data transformations and hardware-specific model compression.
- Explore an unsupervised approach by implementing and evaluating an Isolation Forest solution for comparative anomaly detection.

References

- [1] CWRU Bearing Data Center. *Case Western Reserve University Dataset*, <https://engineering.case.edu/bearingdatacenter>, 2012.
- [2] Universidade Federal do Rio de Janeiro (UFRJ). *Machinery Fault Database (MaFaulDa)*, www02.smt.ufrj.br/~offshore/mfs.
- [3] R. Ibrahim et al., "Fault Detection Based on Vibration Measurements and VAE," *IEEE OJIA*, 2024.
- [4] MathWorks. *What Is an Autoencoder?*, fr.mathworks.com/discovery/autoencoder.html.